



1. Neural Networks and Perceptrons

The “Heart” and “Soul” of Deep Learning - so this is something you must truly understand.

Previous Section:

 [0. Deep Learning](#)

Contents:

[1. Perceptrons — Where Everything Started](#)

[2. Perceptrons — How it is Trained?](#)

[3. Perceptrons — Sample Dataset](#)

[4. From Perceptrons to Neural Networks](#)

[Summary](#)

[References:](#)

In this section, we arrive at the core of everything we’ve been building toward the introductory section.

When people talk about *Deep Learning*, what they are really talking about—at its foundation—are **Neural Networks**. Not as some abstract concept, but as the actual mechanism that allows machines to learn directly from raw data.

Now, you’ve probably heard this before:

“*Neural Networks are inspired by the human brain.*”

And yes, that statement is technically true. But let me be very clear—this is *inspiration*, not replication.

Even today, scientists still don't fully understand why the human brain is so extraordinarily powerful. It can recognize patterns, generalize from very little data, and adapt in ways that our most advanced systems still struggle to match.

So instead of trying to copy the brain exactly, Neural Networks borrow a *simple idea*:

- Information flows through connected units
- Each connection has a level of importance
- And learning happens by adjusting those connections over time

That's it. No biology required.

You can think of a Neural Network as a system that takes input, processes it step by step, and gradually transforms it into something meaningful. It doesn't *start* with understanding—it builds understanding layer by layer.

And this is exactly why Deep Learning works.

- Because instead of asking: "How do we manually define every important feature?"
- We let the network answer a much more powerful question: "What should I pay attention to... if I want to make the right prediction?"

That process—of discovering what matters—is what makes Neural Networks the *heart* of Deep Learning.

But every complex system has a beginning. And for Neural Networks, that beginning is surprisingly simple.

It all starts with a single unit. A model so small, yet so important that it laid the foundation for everything we have today: **Perceptrons**

1. Perceptrons — Where Everything Started

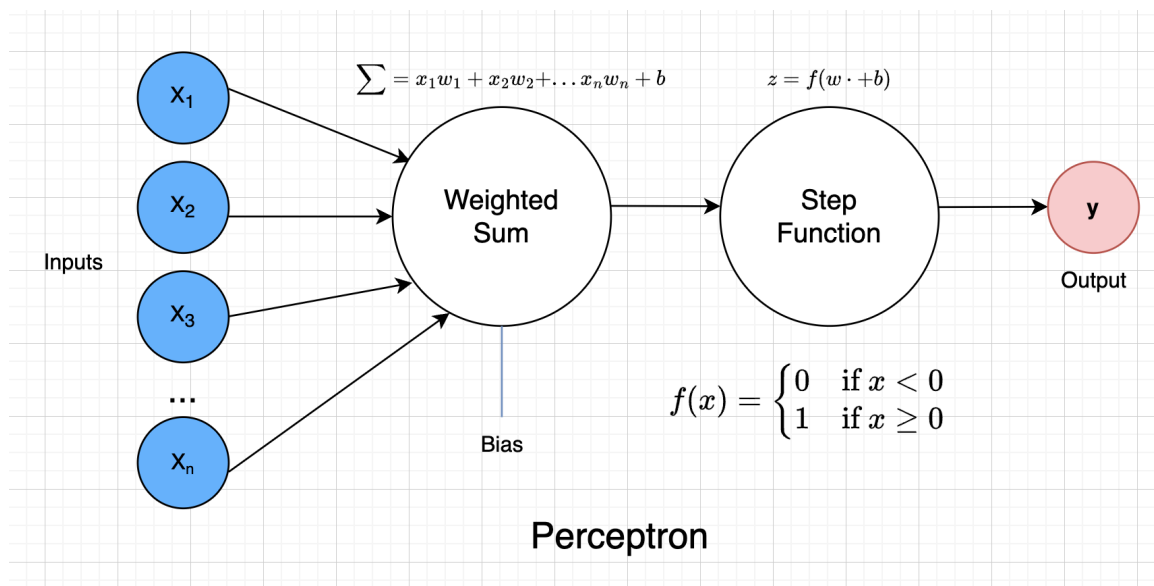
Perceptron is the easiest form of a Neural Network, and I really mean that.

At its core, a perceptron is nothing more than a **single neuron**. No deep stacks of layers, no complex architectures—just one unit making a decision. And yet, this tiny idea is what everything in Deep Learning is built upon.

Its primary purpose? **Binary Classification**.

In other words, it answers questions like: Yes or No; 0 or 1; Spam or Not Spam
That's all it does—but understanding *how* it does this is where things get interesting.

Breaking Down the Structure:



A perceptron takes in **multiple inputs** and produces a **single output**.

You can imagine it like this:

$$x_1, x_2, x_3, \dots, x_n \rightarrow \text{Perceptron} \rightarrow \text{Output}$$

But the real magic happens in the ***middle process***. That's where the perceptron actually "thinks."

- **Weights — Measuring Importance**

First, every input is paired with a weight: $w_1, w_2, w_3, \dots, w_n$

These weights answer a very simple but powerful question: "*How important is this input?*"

- If a weight is large $\rightarrow$ that input matters a lot
- If it's small (or even negative) $\rightarrow$ that input matters less, or pushes the decision the other way

So instead of treating all inputs equally, the perceptron learns **which signals to trust more**.

- **Weighted Sum + Bias — Combining Everything**

Next, we combine all inputs into a single value:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

This is called the **weighted sum**. Now notice the extra term: $+ b \rightarrow$ That's the **bias**.

Think of bias as a way to *shift the decision*.

- Without bias $\rightarrow$ the decision boundary is fixed in a limited way
- With bias $\rightarrow$ the perceptron can move that boundary more freely

If you want an intuition: *The weights shape the decision, the bias shifts it.*

- **Step Function — Making the Final Decision**

At this point, we only have a number z . But we need a **clear answer**: 0 or 1.

So the perceptron passes z through a function called the **step function** (its activation function):

$$\text{output} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

This is the moment the perceptron *decides*.

- If the combined signal is strong enough $\rightarrow$ output 1
- Otherwise $\rightarrow$ output 0

No probabilities, no gray area—just a hard decision.

Putting It All Together, the full process looks like this:

Inputs $\rightarrow$ Weighted Sum ($z = w \cdot x + b$) $\rightarrow$ Step Function $\rightarrow$ Output

And that's it.

A perceptron takes multiple inputs, weighs their importance, combines them into a single value, and then draws a line—literally—between two classes.

It's simple. Almost deceptively simple.

But this one idea—a **weighted sum followed by a decision function**—is the foundation of every Neural Network you'll ever see.

2. Perceptrons — How it is Trained?

So far, we've learned the components.

But now, more questions start to appear than answers:

- Where do these weights even come from?
- How does the perceptron go from inputs to a final decision?
- And more importantly, how does it *improve* over time?

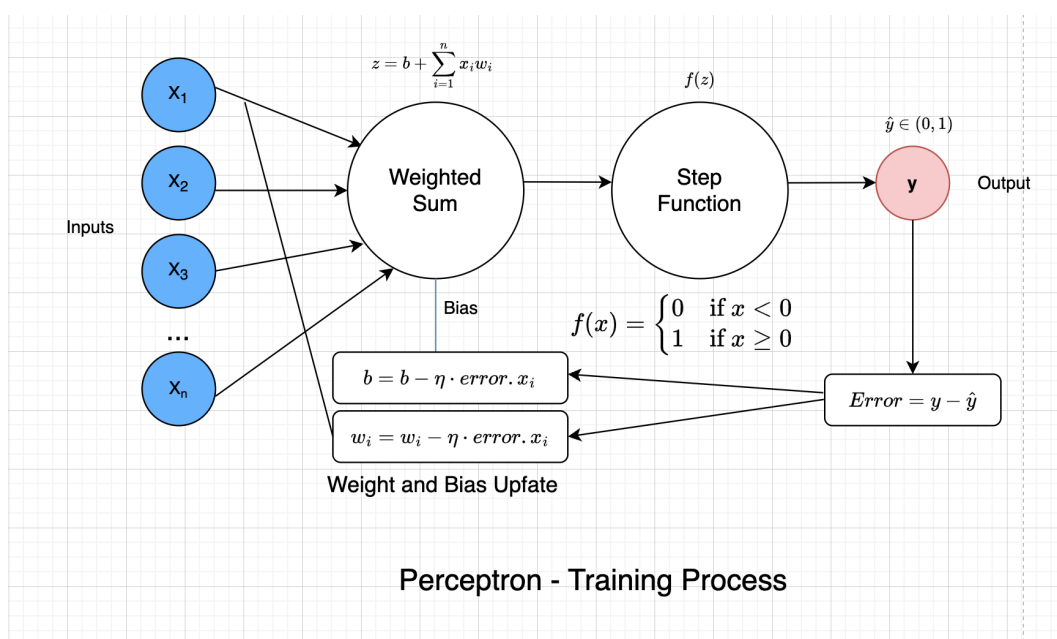
Well, here's the key difference: Unlike many traditional Machine Learning models, a Neural Network doesn't just "learn once and stop." → It **learns while making decisions**.

It predicts → checks if it was wrong → and then *adjusts itself immediately*.

That adjustment is what we call **training**.

The Training Process (Step by Step)

Let's walk through the full process clearly:



Step 1 — Input → We start with our inputs: $x_1, x_2, \dots, x_n$

Step 2 — Initialize Weights → Each input gets a weight: $w_1, w_2, \dots, w_n$

Now here's something interesting:

- You *can* manually set these values
- But in practice, they are usually **randomly initialized**

Because the model will *learn* the correct values anyway.

Step 3 — Weighted Sum → We compute: $z = w \cdot x + b$. Or generally:

$$z = b + \sum_{i=1}^n x_i w_i$$

Step 4 — Activation (Step Function) → $f(z)$ → The perceptron makes a decision:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Step 5 — Compute Error → Now we compare prediction with reality: $\text{error} = \hat{y} - y$

This is the moment of truth.

- If $\text{error} = 0$ → perfect prediction
- If $\text{error} \neq 0$ → the perceptron was wrong

Step 6 — Update Weights and Bias

Now comes the most important part. The perceptron **adjusts itself** using the error:

$$w_i = w_i - \eta \cdot \text{error} \cdot x_i \quad ; \quad b = b - \eta \cdot \text{error}$$

Where: η (eta) is the **learning rate**

Why Do We Need a Learning Rate?

The learning rate controls *how much* we adjust the model each time.

- If it's too large → updates become unstable (jumping around)
- If it's too small → learning becomes painfully slow

So think of it as: *"How big of a step should I take when correcting my mistake?"*

Batch and Batch Size

Each time the perceptron takes an input, makes a prediction, updates its weights...it completes **one learning step**.

Now, here's where things get a bit more structured.

Instead of updating after every single data point, we often group data into what we call a **batch**.

- A **batch** is simply a small subset of the dataset
- The **batch size** tells us how many data points we process before updating the weights

So depending on how we choose it:

- **Batch size = 1** → update after every single example (*very reactive, but noisy*)
- **Batch size = full dataset** → update once after seeing everything (*more stable, but slower to react*)
- **Batch size = something in between** → a balance between speed and stability

You can think of it like this:

Do I learn immediately from every mistake... or do I wait and learn from a group of mistakes together?

Each batch produces one update. And after processing all batches, something bigger happens.

Once the perceptron has gone through **every single data point in the dataset**—whether individually or in batches—it completes one full cycle.

That cycle is called an **epoch**.

Epoch — What Does It Really Mean? → This is the term you'll hear a lot when studying about Deep Learning. And no—it's not just a simple loop.

An **epoch** means: The perceptron has gone through **the entire dataset once**
So:

- One full pass over all training data → **1 epoch**
- Then it starts again → **another epoch**

Each time it repeats this full cycle, it refines its weights a little more.

The perceptron learns through repetition:

Input → Predict → Error → Update → Repeat

Again. And again. And again. Until eventually, it stops making mistakes.

Or at least—makes far fewer of them. And that's the moment where a simple single neuron starts to feel surprisingly intelligent.

3. Perceptrons — Sample Dataset

Alright, this is the part I enjoy the most. Because up until now, everything sounds nice in theory—but Neural Networks are not meant to stay in theory.

They are meant to **learn from actual data**. So let's do exactly that.

Here's a simple dataset for a **Spam Detection problem (10 samples)**:

Feature1 (word freq)	Feature2 (link count)	Feature3 (exclamation count)	Spam (Target)
0.1	1	0	0
0.5	2	1	0
0.9	0	5	1
0.3	3	0	0
0.8	1	3	1
0.7	4	2	1
0.2	0	0	0

Feature1 (word freq)	Feature2 (link count)	Feature3 (exclamation count)	Spam (Target)
0.6	3	4	1
0.4	1	0	0
1.0	5	5	1

Now instead of jumping into complex frameworks... we're going to simulate everything using **plain Python**.

Why? Because I want you to see the learning process—not hide it behind libraries.

```
import numpy as np

# Features (X) and Labels (y)
X = np.array([
    [0.1, 1, 0],
    [0.5, 2, 1],
    [0.9, 0, 5],
    [0.3, 3, 0],
    [0.8, 1, 3],
    [0.7, 4, 2],
    [0.2, 0, 0],
    [0.6, 3, 4],
    [0.4, 1, 0],
    [1.0, 5, 5]
])

y = np.array([0, 0, 1, 0, 1, 1, 0, 1, 0, 1])
```

```
# Initialize weights randomly
np.random.seed(42)
weights = np.random.randn(3)
bias = np.random.randn()

learning_rate = 0.1
```

```
def step_function(z):  
    return 1 if z >= 0 else 0
```

```

epochs = 10
batch_size = 2

for epoch in range(epochs):
    print(f"\nEpoch {epoch + 1}")

    for i in range(0, len(X), batch_size):
        X_batch = X[i:i+batch_size]
        y_batch = y[i:i+batch_size]

        for x_i, y_true in zip(X_batch, y_batch):
            # Forward pass
            z = np.dot(weights, x_i) + bias
            y_pred = step_function(z)

            # Error
            error = y_pred - y_true

            # Update weights and bias
            weights -= learning_rate * error * x_i
            bias -= learning_rate * error

            print(f"Input: {x_i}, Pred: {y_pred}, True: {y_
true}, Error: {error}")

```

Epoch 1

Input: [0.1 1. 0.], Pred: 1, True: 0, Error: 1
 Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1
 Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0
 Input: [0.3 3. 0.], Pred: 1, True: 0, Error: 1
 Input: [0.8 1. 3.], Pred: 1, True: 1, Error: 0
 Input: [0.7 4. 2.], Pred: 0, True: 1, Error: -1
 Input: [0.2 0. 0.], Pred: 1, True: 0, Error: 1
 Input: [0.6 3. 4.], Pred: 1, True: 1, Error: 0
 Input: [0.4 1. 0.], Pred: 1, True: 0, Error: 1
 Input: [1. 5. 5.], Pred: 1, True: 1, Error: 0

Epoch 2

Input: [0.1 1. 0.], Pred: 1, True: 0, Error: 1
 Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1

Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0
Input: [0.3 3. 0.], Pred: 0, True: 0, Error: 0
Input: [0.8 1. 3.], Pred: 1, True: 1, Error: 0
Input: [0.7 4. 2.], Pred: 0, True: 1, Error: -1
Input: [0.2 0. 0.], Pred: 1, True: 0, Error: 1
Input: [0.6 3. 4.], Pred: 1, True: 1, Error: 0
Input: [0.4 1. 0.], Pred: 1, True: 0, Error: 1
Input: [1. 5. 5.], Pred: 1, True: 1, Error: 0

Epoch 3

Input: [0.1 1. 0.], Pred: 1, True: 0, Error: 1
Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1
Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0
Input: [0.3 3. 0.], Pred: 0, True: 0, Error: 0
Input: [0.8 1. 3.], Pred: 1, True: 1, Error: 0
Input: [0.7 4. 2.], Pred: 0, True: 1, Error: -1
Input: [0.2 0. 0.], Pred: 1, True: 0, Error: 1
Input: [0.6 3. 4.], Pred: 1, True: 1, Error: 0
Input: [0.4 1. 0.], Pred: 1, True: 0, Error: 1
Input: [1. 5. 5.], Pred: 1, True: 1, Error: 0

Epoch 4

Input: [0.1 1. 0.], Pred: 1, True: 0, Error: 1
Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1
Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0
Input: [0.3 3. 0.], Pred: 0, True: 0, Error: 0
Input: [0.8 1. 3.], Pred: 1, True: 1, Error: 0
Input: [0.7 4. 2.], Pred: 0, True: 1, Error: -1
Input: [0.2 0. 0.], Pred: 1, True: 0, Error: 1
Input: [0.6 3. 4.], Pred: 1, True: 1, Error: 0
Input: [0.4 1. 0.], Pred: 1, True: 0, Error: 1
Input: [1. 5. 5.], Pred: 1, True: 1, Error: 0

Epoch 5

Input: [0.1 1. 0.], Pred: 0, True: 0, Error: 0
Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1
Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0
Input: [0.3 3. 0.], Pred: 0, True: 0, Error: 0
Input: [0.8 1. 3.], Pred: 1, True: 1, Error: 0
Input: [0.7 4. 2.], Pred: 0, True: 1, Error: -1
Input: [0.2 0. 0.], Pred: 1, True: 0, Error: 1
Input: [0.6 3. 4.], Pred: 1, True: 1, Error: 0
...

(You can run this code yourself to see the full result)

Understanding What's Happening Behind the Code

Before we move any further, I want to slow this down a bit. Because what we just ran is not just "some code"—it's the perceptron **learning in real time**.

Let's break it apart properly. **What Are These Components, Really?**

- X is our input: x_1, x_2, x_3
 - But here's something subtle: Each x_i is not just a number—it's a **vector of features**.
 - So when you see: $[0.1, 1, 0]$
 - That's one data point with three features.
- y is the output (label)
 - And this is important: The model does **not** know y when making predictions.
 - It only sees y **after** it predicts—so it can measure how wrong it was.
- `np.random.seed(42)`
 - This line looks small... but it matters a lot. It ensures that the **random weights and bias are the same every time** you run the code.
 - And why 42? → Well... apart from being "*the meaning of life*"—there's no real reason.
 - You can pick any number. It's just a way to make randomness... reproducible.

Now Let's Talk About Batch Size (The Part Most People Overlooked)

We set: `batch size = 2` → So what does that actually mean here?

It means:

The model looks at **2 data points** and then updates its weights and bias. Then it moves to the next 2. And repeats → Until it has seen the entire dataset.

Imagine our inputs like this: $x_1, x_2, x_3, x_4, \dots, x_n$

Now the process becomes:

- **Batch 1:** Take $x_1, x_2 \rightarrow$ Predict $\rightarrow$ Update weights & bias
- **Batch 2:** Take $x_3, x_4 \rightarrow$ Predict $\rightarrow$ Update again
- **Batch 3:** Take $x_5, x_6 \rightarrow$ Repeat
- ...until all data is used.

At that moment—**one epoch is completed.**



Here, each step is called a **batch**. Since we have 10 samples in total, and each batch looks at 2 of them. So, 5 batches is required to fully learn the dataset in a single epoch.

One More Important Detail (This Matters a Lot): When we move on to the **next epoch**, something very important happens: We **do not reset the weights and bias**. Instead, we continue using the **latest updated weights and bias**

This means learning is **cumulative**.

- Epoch 1 $\rightarrow$ starts with random weights and bias
- Epoch 2 $\rightarrow$ starts with *improved* weights and bias
- Epoch 3 $\rightarrow$ starts with *even better* weights and bias
- And so on...

Each epoch is not starting over—it's **refining what was already learned**.

Also, a quick reminder:

- The weights are not just numbers
- They form a **vector** $w = [w_1, w_2, w_3]$

That's why we use the **dot product**: $w \cdot x$

Because we are combining two vectors: Input vector x and the Weight vector w

So every time the model updates: It is slightly adjusting this weight vector.

Reshaping the decision boundary. Becoming more accurate over time

And only after understanding this, can we truly appreciate what the output is showing us.

What You're Seeing in the Output proves it all

Take a look at this:

Epoch 1

Input: [0.1 1. 0.], Pred: 1, True: 0, Error: 1

Input: [0.5 2. 1.], Pred: 1, True: 0, Error: 1

This is literally the perceptron *thinking*:

- "I predict 1" → "Oh... that was wrong" → "Let me adjust myself"
- And because `batch size = 2`: These adjustments happen **after every two data points**

Then it continues:

Input: [0.9 0. 5.], Pred: 1, True: 1, Error: 0

- Now it gets something correct → no adjustment needed.



A Small but Important Detail (Don't Miss This)

Now let me ask you something.

- What if in a batch of two, **one prediction is correct**?
- What if **both predictions are correct**?

What happens then? The answer is surprisingly simple: The **error becomes 0**.

And when the error is 0 → The weight update becomes 0 → The bias update becomes 0 → Which means, **nothing changes**

But here's the subtle insight: Even when nothing changes, the perceptron is still **going through the update step**. It's still *trying* to adjust. It's just that multiplying by 0 **cancels everything out**.

So technically, the perceptron is **always updating**. It's just that sometimes—when it's already correct—those updates have **no effect at all**.

And this is actually a very good sign. Because it means:

- The model is no longer making mistakes
- The weights have reached a state where they work

- Learning naturally slows down

In other words, when nothing changes anymore, it's not because the model stopped learning—it's because it has **nothing left to fix**

Now, look at the Learning Itself

At the beginning:

- Predictions are honestly terrible
- The model is guessing more than understanding

But as epochs go by:

```
Epoch 10
Input: [0.1 1. 0. ], Pred: 0, True: 0, Error: 0
...
Input: [0.4 1. 0. ], Pred: 0, True: 0, Error: 0
```

Something changes.

- Errors start disappearing
- Predictions align with reality
- The model becomes consistent

Nothing magical happened. No rules were hardcoded. No human told it what "spam" means.

It simply: Made a guess → Measured how wrong it was → Adjusted itself → Repeated → Again. And again → Until the mistakes almost disappeared.

And this is why I always love to showcase the real workflow. Because right here—you're not just *learning about* Neural Networks. You're watching one... **learn.**

4. From Perceptrons to Neural Networks

Now, let's zoom out for a moment. Everything we've done so far—weights, bias, step function, training— all of it revolves around a **single perceptron**.

And that's exactly the point: ***A perceptron is just one neuron***

The simplest possible version of a Neural Network. The most basic building block.

So what happens when we go beyond that? We don't change the idea. We **scale it**.

Instead of one neuron, we use **many neurons**. And when neurons start working together, something new emerges: **Neural Networks**

What Changes in Neural Networks? At a high level, two major ideas are introduced:

- **Layers:** Instead of a single unit, neurons are organized into layers:
 - **Input Layer** → receives data
 - **Hidden Layers** → process and transform data
 - **Output Layer** → produces the final prediction

So instead of: Input → Output

We now have: Input → Hidden Layers → Output

Each layer builds on the previous one—learning more complex patterns step by step.

- **Activation Functions**
 - In perceptrons, we used the **step function**—a very strict, binary decision.
 - But in real Neural Networks, we use more flexible activation functions.
 - Why? Because the world is not just 0 or 1. We need functions that can: Capture subtle patterns; Handle continuous values; Allow smoother learning (You'll see **this again in future section, so be prepared**)
 - But for now, just remember this: Activation functions decide **how a neuron responds**

And don't worry—we'll go much deeper into both layers and activation functions in future posts. Right now, I just want you to see the transition clearly:

- **Perceptron** → **One neuron**
- **Neural Network** → **Many neurons working together**

Summary

Neural Networks are inspired by the human brain. And I want to emphasize this again—**inspired**, not imitated, not replicated, and definitely not “better than it”. We simply borrowed an idea: connections, signals, and learning through adjustment. That’s all.

A **Neural Network** refers to any system built from neurons. And a **Perceptron**? That’s where everything begins.

| *If a Neural Network is a system, then a perceptron is **Level 1** → One neuron
→ One decision.*

And from something that simple, we built the entire world of Deep Learning.

References:

<https://www.geeksforgeeks.org/deep-learning/what-is-perceptron-the-simplest-artificial-neural-network/>

<https://www.quantstart.com/articles/introduction-to-artificial-neural-networks-and-the-perceptron/>

Next Section:

 2. Deep Neural Networks